

Ranges Without Compromises

Designing for Simplicity, Performance,
and Composability

OLEKSANDR BACHERIKOV

Challenging problems
Competitive aspect

Making algorithms
reusable and composable

Range views
promised a solution for
sequence algorithms

Alternatives to “raw loops”: before range views

4

Raw loops

```
void process(std::vector<int> const& v) {  
    for (size_t i = 0; i < v.size() - 2; ++i) {  
        if (check(fn(v[i]))) {  
            std::cout << fn(v[i]) << ' ';  
        }  
    }  
}
```

Algorithms

```
void process(std::vector<int> const& v) {  
    std::span span{v.begin(), v.end() - 2};  
    using T= decltype(fn(std::declval<int>()));  
    std::vector<T> transformed; extra allocation  
    std::ranges::transform(span,  
        std::back_inserter(transformed),  
        fn); multiple  
    std::ranges::copy_if( passes  
        transformed,  
        std::ostream_iterator<T>{  
            std::cout, " "},  
        check);  
}
```

Alternatives to “raw loops”: with range views

Raw loops

```
void process(std::vector<int> const& v) {  
    for (size_t i = 0; i < v.size() - 2; ++i) {  
        if (check(fn(v[i]))) { what if size < 2?  
            std::cout << fn(v[i]) << ' ';  
        } duplicate call  
    }  
}
```

Range views

```
void process(std::vector<int> const& v) {  
    using namespace std::views;  
    for (const auto& x : v  
        | drop_last(2)  insert  
        | transform(fn)  here  
        | filter(check)) {  
        std::cout << x << ' ';  
    }  
}
```

1. Correctness
2. Composability
3. Clarity

Existing ranges libraries

- [range-v3](#) by Eric Niebler, [Ranges for the Standard Library](#) - CppCon 2015
- [rangeless](#) by Alex Astashyn, [The Surprising Limitations of C++ Ranges Beyond Trivial Cases](#), 2019
- [pipes](#) by Jonathan Boccara,
[Pipes: How Plumbing Can Make Your C++ Code More Expressive](#) - CppCon 2020
- [think-cell-library](#) by ThinkCell, [Better C++ Ranges - Arno Schödl](#) - C++Now 2021
[Missing \(and Future?\) C++ Range Concepts - Jonathan Müller](#) - C++Now 2025
- [rivers](#) by Barry Revzin,
[Iterators and Ranges: Comparing C++ to D, Rust, and Others](#) - CPPP 2021
- [bellevue](#) by Nicolai Josuttis,
[Belle Views on C++ Ranges, their Details and the Devil](#) - Meeting C++ 2022
- [Flux](#) by Tristan Brindle, [Iteration Revisited: A Safer Iteration Model for Cpp](#) - CppCon 2023,
[Faster, Safer, Better Ranges](#) - C++ on Sea 2025
- [SPL](#) by John Bandela, previously known as Rappel,
[Rappel: Compose Algorithms, Not Iterators - Google's Alternative to Ranges](#) - C++Now 2024
- [daisychains](#) by Daisy Hollman,
[Ranges++: Are Output Range Adaptors the Next Iteration of C++ Ranges?](#) - CppCon 2024
- [transrangers](#) by Joaquín M López Muñoz, [Push is faster](#) - Using std::cpp 2025

Examples from different libraries

```
using namespace std::views;
for (int n : std::vector{1, 2, 3, 4, 5}
    | transform([](int n) { return n * 2; })
    | filter([](int n) { return n % 4 == 0; })) {
    std::cout << n << '\n';
}
```

```
rvr::from(std::vector{1, 2, 3, 4, 5})
    .map([](int n) { return n * 2; })
    .filter([](int n) { return n % 4 == 0; })
    .for_each([](int n) { std::cout << n << '\n'; });
```

```
using namespace transrangers;
filter([](int n) { return n % 4 == 0; },
    transform([](int n) { return n * 2; },
    all(std::vector{1, 2, 3, 4, 5}))) (
    [](auto p) { std::cout << *p << '\n'; }
);
```

```
std::vector{1, 2, 3, 4, 5}
    >=> pipes::transform([](int n) { return n * 2; })
    >=> pipes::filter([](int n) { return n % 4 == 0; })
    >=> pipes::for_each([](int n) {
        std::cout << n << '\n';
    });
```

```
flux::for_each(
    flux::from(std::vector{1, 2, 3, 4, 5})
        .map([](int n) { return n * 2; })
        .filter([](int n) { return n % 4 == 0; })),
    [](int n) { std::cout << n << '\n'; }
)
```

```
rpl::Apply(
    std::vector{1, 2, 3, 4, 5},
    rpl::Transform([](int n) { return n * 2; }),
    rpl::Filter([](int n) { return n % 4 == 0; }),
    rpl::ForEach([](int n) { std::cout << n << '\n'; })
);
```

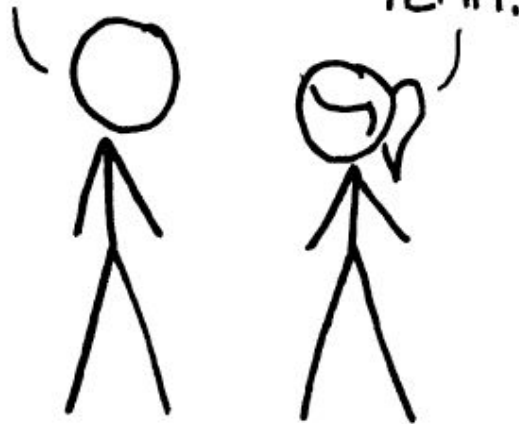
HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)

8

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

1. Iterators vs indices
2. External vs internal iteration
 - a. Simplicity and performance
 - b. Composability
3. Internal iteration design space

Example 1 by Tristan Brindle

10

Iterators

```
auto iter = std::end(rng);  
do_something(*iter);
```

Dereferencing a past-the-end iterator
is UB

Indices

```
auto idx = std::size(vec);  
do_something(vec.at(idx));
```

Reading via a past-the-end index
is bounds checked

Note: only if we use at not operator []

Example 2 by Tristan Brindle

Iterators

```
auto iter = std::end(rng);  
++iter;
```

Incrementing a past-the-end iterator is UB
So is decrementing a **begin()** iterator
So are out-of-bounds random-access
jumps

Indices

```
auto idx = std::size(vec);  
++idx;
```

Incrementing a past-the-end index is fine
(Assuming unsigned indices or
checked operations on signed indices)

Example 3 by Tristan Brindle

Iterators

```
auto min_iter(std::vector<int> const& v) {  
    return std::min_element(v.begin(), v.end());  
}  
  
auto iter = min_iter(get_vector());  
std::cout << "Minimum: " << *iter << std::endl;
```

Dangling iterators can occur easily
Any operation on a dangling iterator
is UB

Indices

```
size_t min_idx(std::vector<int> const& v) {  
    ...  
}  
  
auto idx = min_idx(get_vector());  
// std::cout << "Minimum: " << ?? << std::endl;
```

“Dangling indices” are safe by design

Note: `std::ranges::min_element`
is also safe by design

Example 4 by Tristan Brindle

Iterators

```
auto iter = vec.begin();  
vec.push_back(99);  
while (iter != vec.end()) {  
    ...  
}
```

Iterators can become dangling
in non-obvious ways

Indices

```
size_t idx = 0;  
vec.push_back(99);  
while (idx < vec.size()) {  
    ...  
}
```

“Index invalidation” problems are not UB
Caught by bounds check at next
attempted read

Let's pass a sequence into each iteration operation!

14



Cppcon 2023
The C++ Conference



October 01 - 06

+

23



Tristan Brindle

Iteration Revisited

ITERATION REVISITED

26

ITERATION OPERATIONS WITH FLUX

- ▶ `flux::first(seq)` returns a *cursor*, which represents a sequence position
 - ▶ For contiguous sequences, this is just an integer index
- ▶ `flux::is_last(seq, cur)` returns whether the cursor is past-the-end
- ▶ `flux::read_at(seq, cur)` accesses the element at the cursor position
- ▶ `flux::inc(seq, cur)` increments the cursor

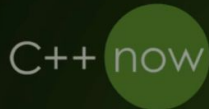
Video Sponsorship Provided By:



think-cell

Let's pass a sequence into each iteration operation!

15



Better C++ Ranges

think-cell

Beyond C++20 Ranges:

Index Concept

Index

- Like iterator
- But all operations require its range object

```
template<typename Base, typename Func>
struct index_range {
    ...
    using Index=...;
    Index begin_index() const;
    Index end_index() const;
    void increment_index( Index& idx ) const;
    void decrement_index( Index& idx ) const;
    reference dereference( Index const& idx ) const;
    ...
};
```



Arno Schödl

Nested filters' iterator size explosion

16

C++ now

Better C++ Ranges

Again: How does iterator look like of

think-cell

```
views::filter(m_func3)(views::filter(m_func2)(views::filter(m_func1, ...))) ?
```

```
m_rng3  
m_it3  
  m_rng2  
  m_it2  
    m_rng1  
    m_it1
```

- Still not insanely great...

C++ now 2024
Aspen, Colorado

CppNow.org

Video Sponsorship Provided By

millennium
think-cell



CUBIC GROWTH

- $144 + ((N+1)(N+2)(N+3)/6 + 2N) * 8$ bytes
- h/t Richard Smith

Rappel: Compose Algorithms, Not Iterators
Google's Alternative to Ranges

John Bandela, Chris Philip



Arno Schödl

JET
BRAINS

Bloomberg

Engineering

Super-Efficient Range Adaptors With Indices

think-cell

Index-based filter_view

```
template<typename Base, typename Func>
struct filter_view {
    Func m_func;
    Base& m_base;

    using Index=typename Base::Index;
    void increment_index( Index& idx ) const {
        do {
            m_base.increment_index(idx);
        } while( idx!=m_base.end_index()
            && !static_cast<bool>(m_func(m_base.dereference_index(idx)))
        );
    }
};
```



Arno Schödl

Graphs in competitive programming

```
1 #include <stdio>
2 #include <vector>
3 using namespace std;
4
5 int main() {
6     int n, m; // number of vertices and edges
7     scanf("%d%d", &n, &m);
8     vector<vector<int>> g(n); // adjacency list graph
9     for (int i = 0; i < m; ++i) {
10         int a, b; // edge endpoints
11         scanf("%d%d", &a, &b);
12         g[a].push_back(b);
13         g[b].push_back(a);
14     }
15     // graph algorithm goes here
16 }
```

Indices are more efficient to store:

`sizeof(int) < sizeof(int*)`

Indices can be simply deserialized,
while pointers need re-adjustment

We don't want to invalidate stored edges
when new vertices or edges are added



Arthur O'Dwyer

Stuff mostly about C++

[Recent](#)

[Date](#)

[Tag](#)

[Training](#)

[About](#)

Pitfalls and decision points in implementing `const_iterator`

This antipattern keeps coming up, so here's the blog post I can point people to. Today we're talking about `const_iterator`. First of all — you know this — `const_iterator` is different from `iterator const`, in exactly the same way that `const int *` is different from `int *const`.

Indices make `const_iterator` redundant

`const` is important for reading elements:

- `T& std::vector<T>::at(size_type index);`
- `T const& std::vector<T>::at(size_type index) const;`

all other iteration operations should only use `const` range (like `increment`)

We should be bold!

Random-access containers already provide

`vector[index]`

`vector.at(index)`

I wouldn't add

`read_at(seq, cur)`

`range.dereference(index)`

Indices solve these problems

1. Dangling and invalidated iterators
2. Some other UB
3. Bounds checking
4. Iterator size
5. **const_iterator**
6. Stable serialization (for random-access containers)

Indices create these problems

1. Inconvenience of passing the sequence into each iteration operation

Is this a big problem?

- User code should use high level algorithms, which hide these operations
- `array[index]` syntax is widely accepted

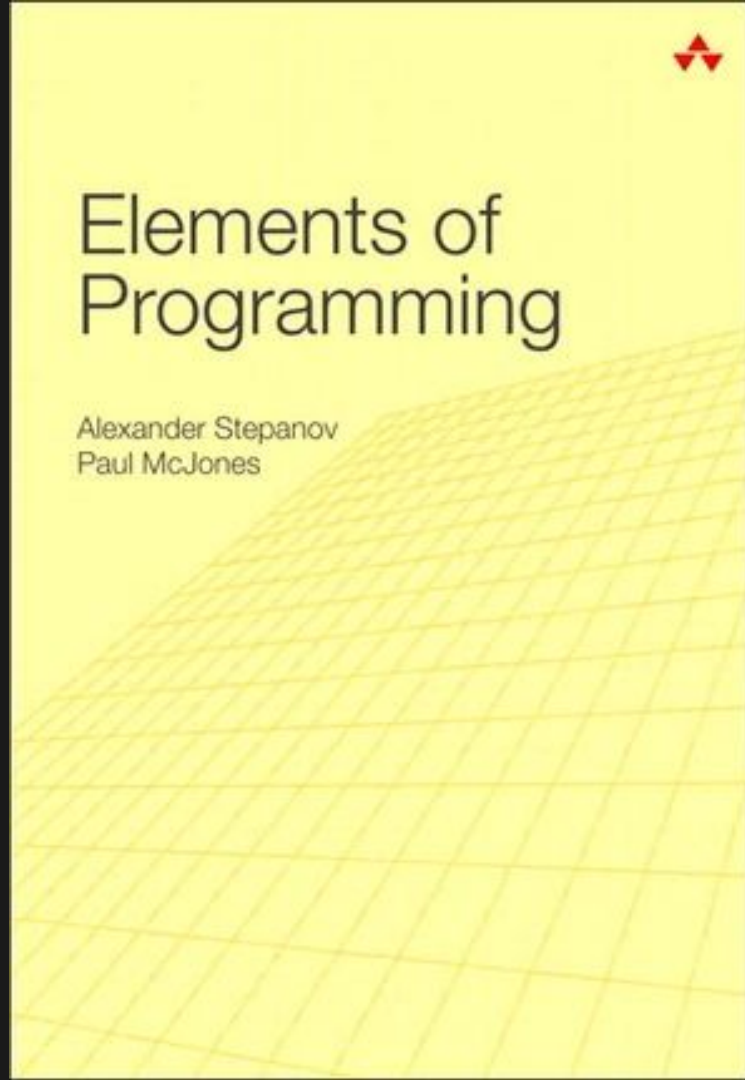
Listening to wise people

“Every object has an address:
an integer index into computer memory”

Alexander Stepanov

Elements of Programming, 2009

The first sentence of Chapter 6: Iterators



1. Iterators vs indices
2. External vs internal iteration
 - a. Simplicity and performance
 - b. Composability
3. Internal iteration design space

Internal Iteration

think-cell

- Producer calls consumer to offer new element
- example: `for_each_xxx, "visitor"`



Producer is at bottom of stack

- ... all the advantages of being bottom of stack
- Consumer is at top of stack
- ... all the disadvantages of being top of stack



Tristan Brindle

Iteration Revisited

ITERATION REVISITED

42

INTERNAL ITERATION

- ▶ The Flux sequence protocol has an extra, optional customisation point: `for_each_while(predicate)`
- ▶ This instructs the sequence to perform *internal iteration* until the predicate returns false
- ▶ Many sequence adaptors can perform internal iteration more efficiently than by a check/read/advance loop
- ▶ Many algorithms can be written in terms for `for_each_while` (e.g. `fold()`, `find_if()`, `min()`)
- ▶ => Flux pipelines can in many cases generate more efficient code than equivalent C++20 Ranges pipelines

Video Sponsorship Provided By:

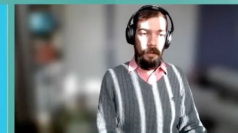


Rappel: Compose Algorithms, Not Iterators
Google's Alternative to Ranges

John Bandela, Chris Philip

Basic Internal Iteration Structure

```
template <stream S>
void print_all(S stream) {
    stream.for_each([](auto&& elem) {
        fmt::print("{}\n", elem);
    });
}
```



Barry Revzin



Science to the CORE

RAPPEL

alternative to `std::ranges` for algorithm

- Based on the eager model of `<algorithm>`
- Passes each value through the series of transformations using continuation passing style.

C++ USING STD::CPP 2025



Some push and

Push

- Reactive programming
- Continuation-passing style
- Senders/receivers (in a sense)

Pull

- STL iterators
- C++ ranges
- SQL (in a sense)

External and internal iteration definition

External iteration:
range-based for loop in C++

```
auto iter = begin(range);  
auto last = end(range);  
while (iter != last) {  
    process(*iter);  
    ++iter;  
}
```

- User code contains multiple API calls and a loop itself
- User code calls processing function

Internal iteration

```
for_each(range, process);
```

- User code calls a single **for_each**, passing the processing function
- **for_each** calls it in internal loop

External iteration: transform

```
template<input_range View, move_constructible Fn>
class transform_view :
public view_interface<transform_view<View, Fn>> { 1
    template<bool Const> 2, 3
    class iterator {
        using Parent = maybe_const<Const, transform_view>;
        using Base = maybe_const<Const, View>;
        Parent* parent = nullptr;
        iterator_t<Base> current = iterator_t<Base>();

    public:
        using iterator_concept = /* ... */;
        using value_type = remove_cvref_t<
            invoke_result_t<Fn&, range_reference_t<Base>>>;
        using difference_type = range_difference_t<Base>;

        iterator() = default;
        iterator(Parent&, iterator_t<Base>);
        iterator(iterator<not Const>) requires Const;

        reference operator*() const {
            return invoke(*parent->fn, *current); }
        iterator& operator++() { ++current; return *this; }
        iterator operator++(int) {
            auto t = *this; ++*this; return t; }
        bool operator==(iterator const& r) {
            return current == r.current; }
    };
```

```
template<bool Const> 4, 5
class sentinel {
    using Base = maybe_const<Const, View>;
    sentinel_t<Base> end = sentinel_t<Base>();

    public:
        sentinel() = default;
        sentinel(sentinel_t<Base>);
        sentinel(sentinel<not Const>) requires Const;

        template<bool OtherConst>
        bool operator==(iterator<OtherConst> const& r) {
            return end == r.current; }
    };

    movable_box<Fn> fn;
    View base = View();

    public:
        transform_view(View, Fn);

        iterator<false> begin();
        sentinel<false> end();
        iterator<false> end() requires common_range<View>;
        iterator<true> begin() const;
        sentinel<true> end() const;
        iterator<true> end() const
            requires common_range<View const>;
    };
```

Custom range view implementation is too complex!

There are helper utilities:

- [std::ranges::view_interface](#)
- [std::iterator_interface](#) (C++29?)
- [ac::iterator_interface](#)
- More stuff in [Boost.STLInterfaces](#)

They reduce implementation complexity only by a constant factor

Internal iteration over an array

Just a single **for_each_while** function to implement:

```
1 template<typename Element, unsigned Length>
2 bool for_each_while(Element (&array)[Length], auto&& sink) {
3     for (unsigned i = 0; i < Length; ++i)
4         if (not sink(array[i]))
5             return false;
6     return true;
7 }
```

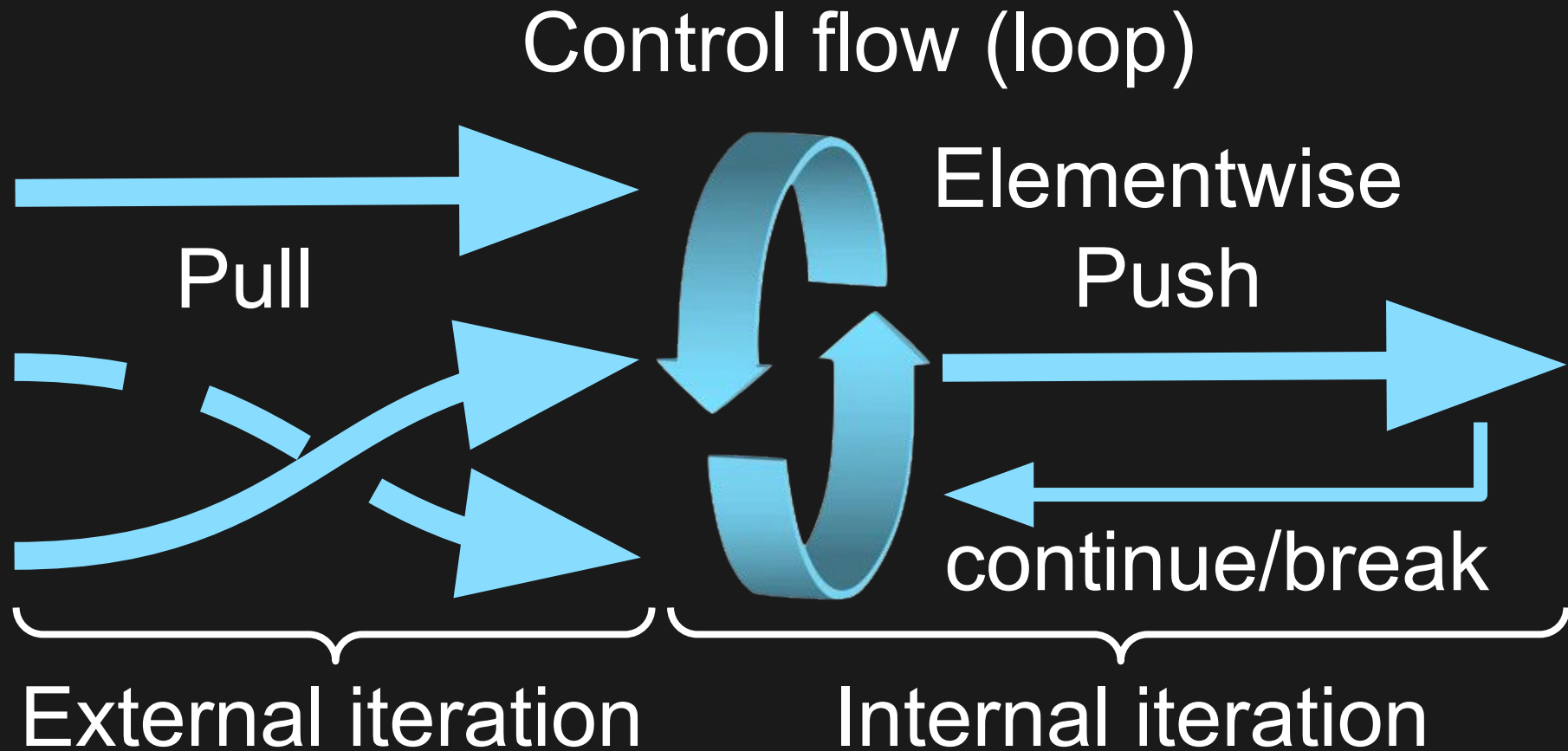
It takes a sink predicate
(callback or receiver, not filter)

invokes it on all the array elements
until the predicate tells it to stop (by returning **false**)

and returns **false** if short circuiting occurred, to support:
all_of, **any_of**, **none_of**, **contains**

In the ideal world...

```
1 template<typename Element, unsigned Length>
2 auto for_each(Element (&array)[Length]) {
3     for (unsigned i = 0; i < Length; ++i)
4         yield array[i];
5 }
```



Pull and push model for an array

```
1 template<typename Element, unsigned Length>
2 bool for_each_while(Element (&array)[Length], auto&& sink) {
3     for (unsigned i = 0; i < Length; ++i)
4         if (not sink(array[i]))
5             return false;
6     return true;
7 }
```

Diagram illustrating the pull and push models in the `for_each_while` function:

- The `push` model is represented by the `return false;` statement (line 5), which is reached when the sink function returns `false`.
- The `pull` model is represented by the `sink(array[i])` call (line 4), which is reached when the sink function is called.

filter

34

```
using Array = int const (&)[10000];
```

```
constexpr auto is_even = [](int x) { return x % 2 == 0; };
```

can't use range-based for loop

```
int raw(Array a) {  
    int sum = 0;  
    for (int x : a)  
        if (is_even(x))  
            sum += x;  
    return sum;  
}
```

```
int internal(Array a) {  
    int sum = 0;  
    for_each_while(  
        a | filter(is_even),  
        [&](int x) {  
            sum += x;  
            return true;  
        })  
    );  
    return sum;  
}
```

boilerplate
(avoidable)

```
#include <ranges>  
  
int external(Array a) {  
    int sum = 0;  
    using std::views::filter;  
    for (int x :  
        a | filter(is_even)  
    )  
        sum += x;  
    return sum;  
}
```

filter implementation

```
1 template<typename Input, typename Predicate>
2 struct filter_view {
3     Input input;
4     Predicate filter;
5
6     friend bool for_each_while(filter_view view, auto&& sink) {
7         return for_each_while(view.input, [&](auto&& x) {
8             if (view.filter(x))
9                 return sink(x);
10            else
11                return true; // continue
12        });
13     }
14 };
```

hidden friend idiom

iteration over input with a new sink

vs 171 lines [range/v3/view/filter.hpp](#)

+ 209 lines [range/v3/view/remove_if.hpp](#)

filter implementation for piping

```
1 template<typename Predicate>
2 struct filter {
3     Predicate predicate;
4
5     template<typename Input>
6     friend auto operator|(Input&& input, filter filter) {
7         return filter_view<Input, Predicate>{
8             static_cast<Input&&>(input), // std::forward
9             static_cast<Predicate&&>(filter.predicate),
10         };
11     }
12 };
```

Now we compile the code...

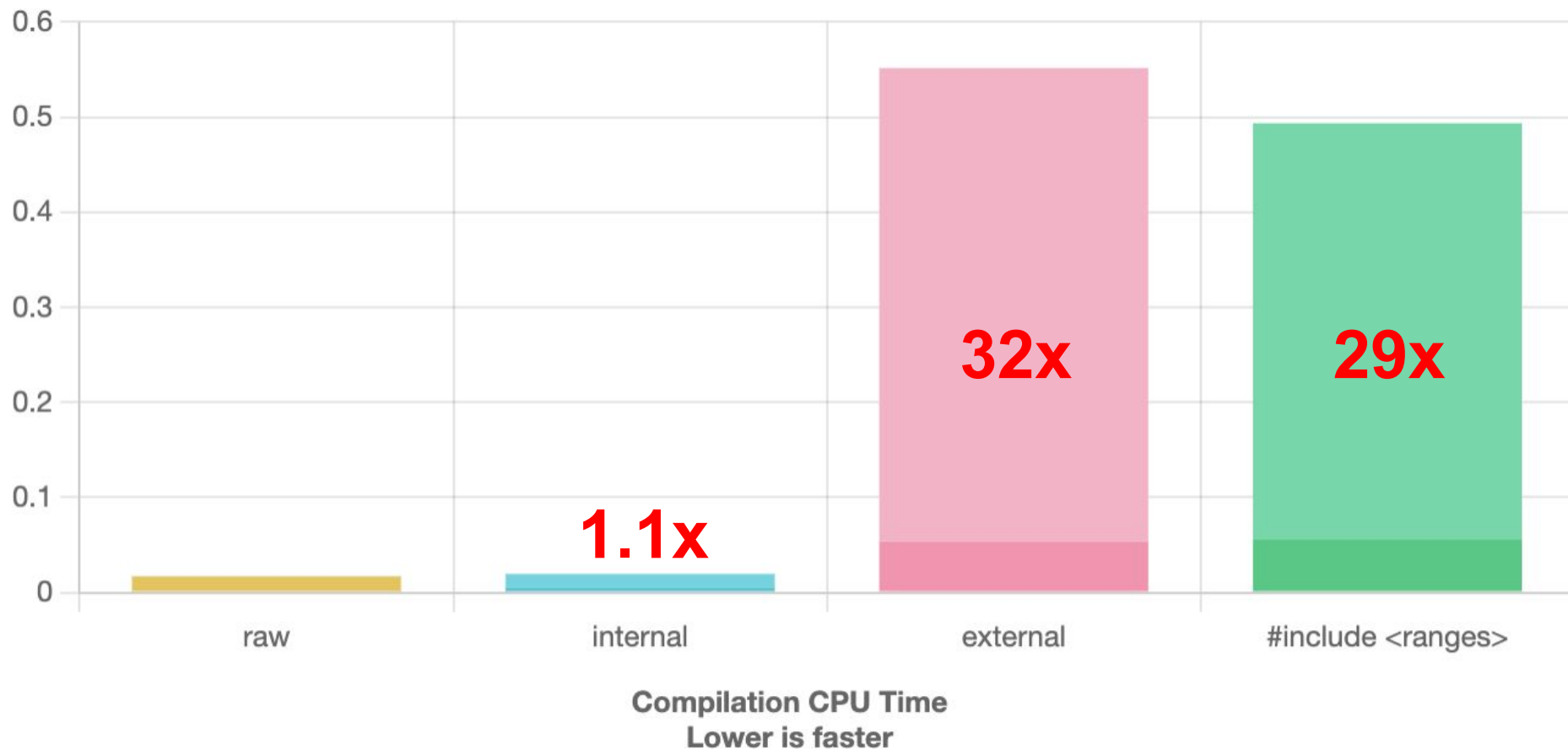
```
using Array = int const (&)[10000];
```

```
constexpr auto is_even = [](int x) { return x % 2 == 0; };
```

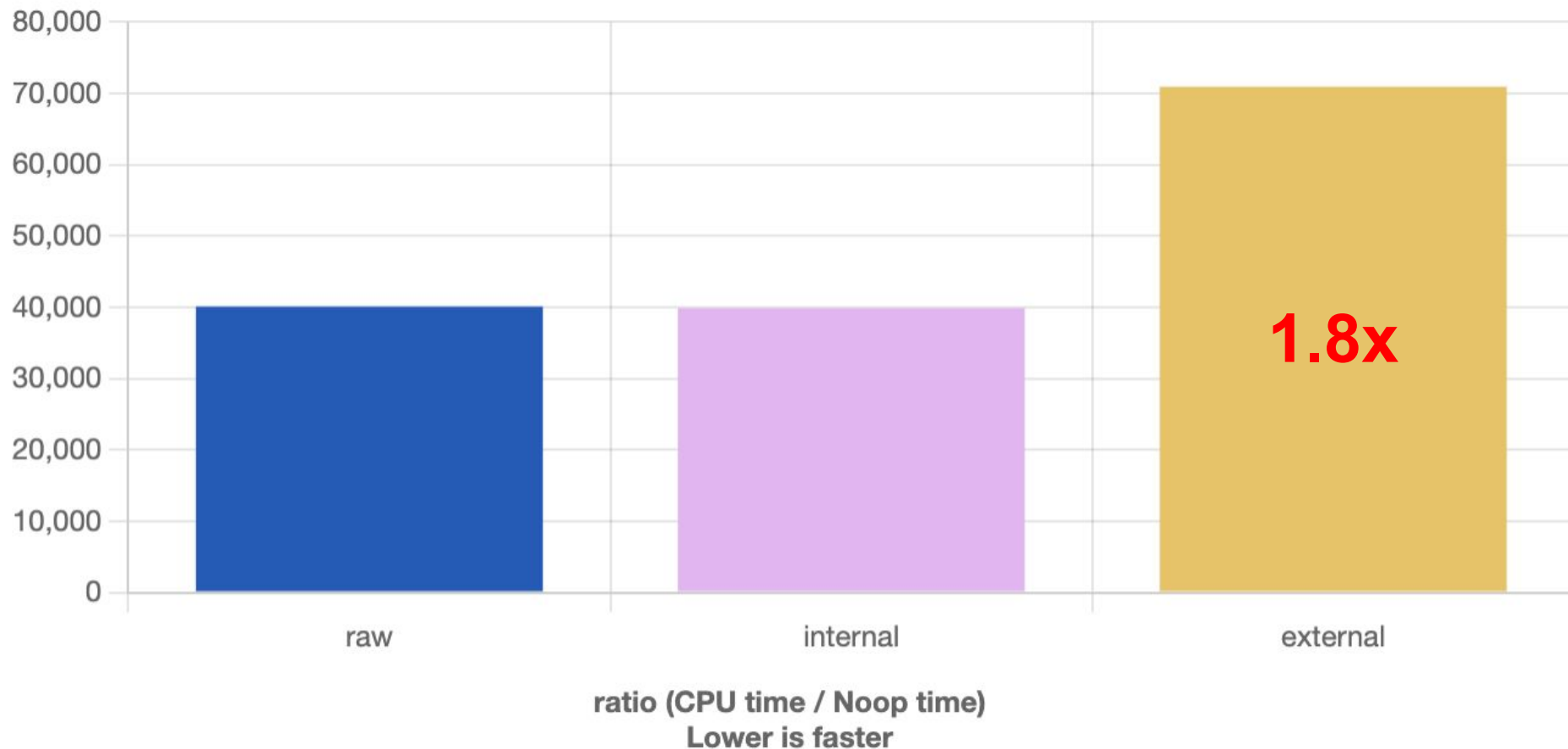
```
int raw(Array a) {  
    int sum = 0;  
    for (int x : a)  
        if (is_even(x))  
            sum += x;  
    return sum;  
}
```

```
int internal(Array a) {  
    int sum = 0;  
    for_each_while(  
        a | filter(is_even),  
        [&](int x) {  
            sum += x;  
            return true;  
        })  
    );  
    return sum;  
}
```

```
#include <ranges>  
  
int external(Array a) {  
    int sum = 0;  
    using std::views::filter;  
    for (int x :  
        a | filter(is_even)  
    )  
        sum += x;  
    return sum;  
}
```

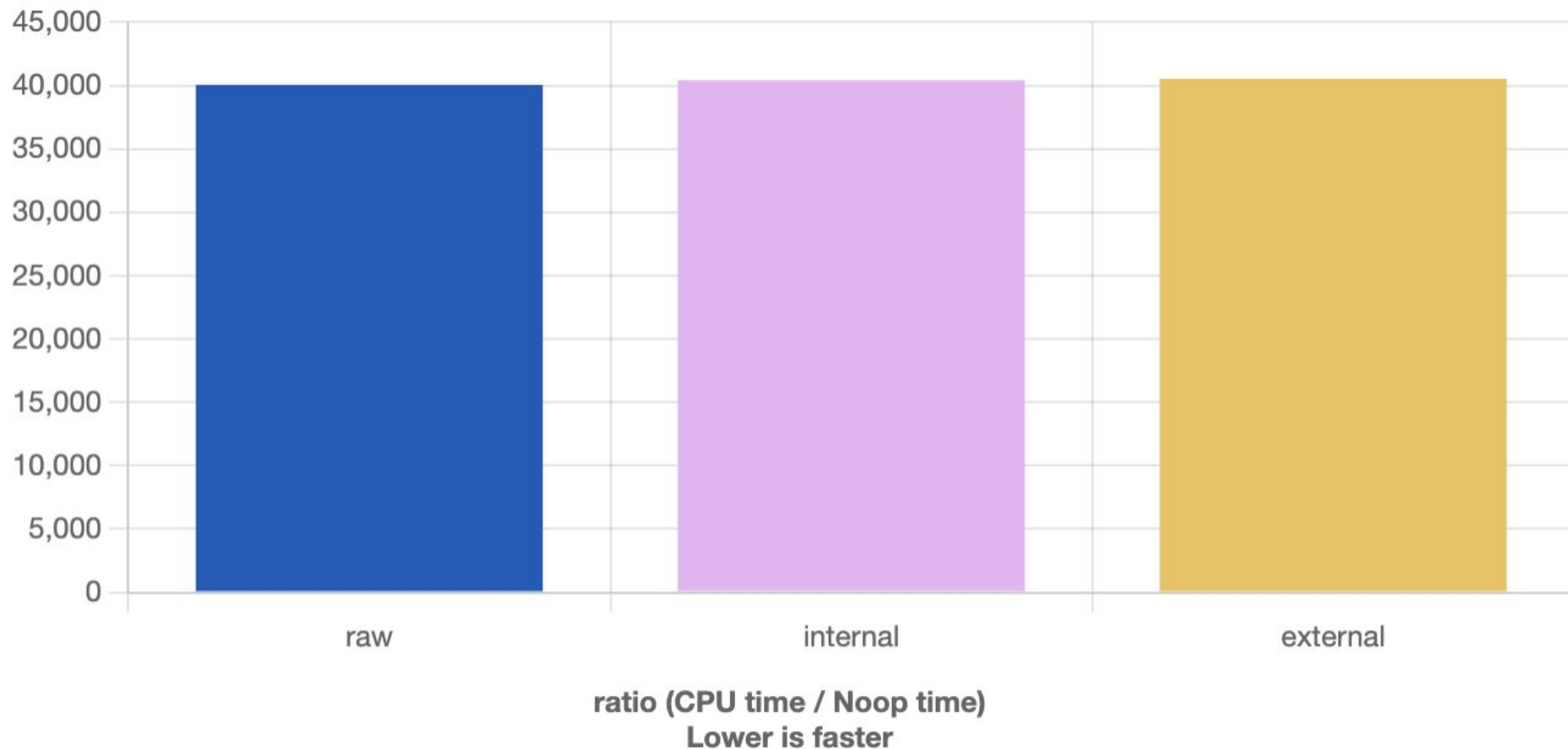


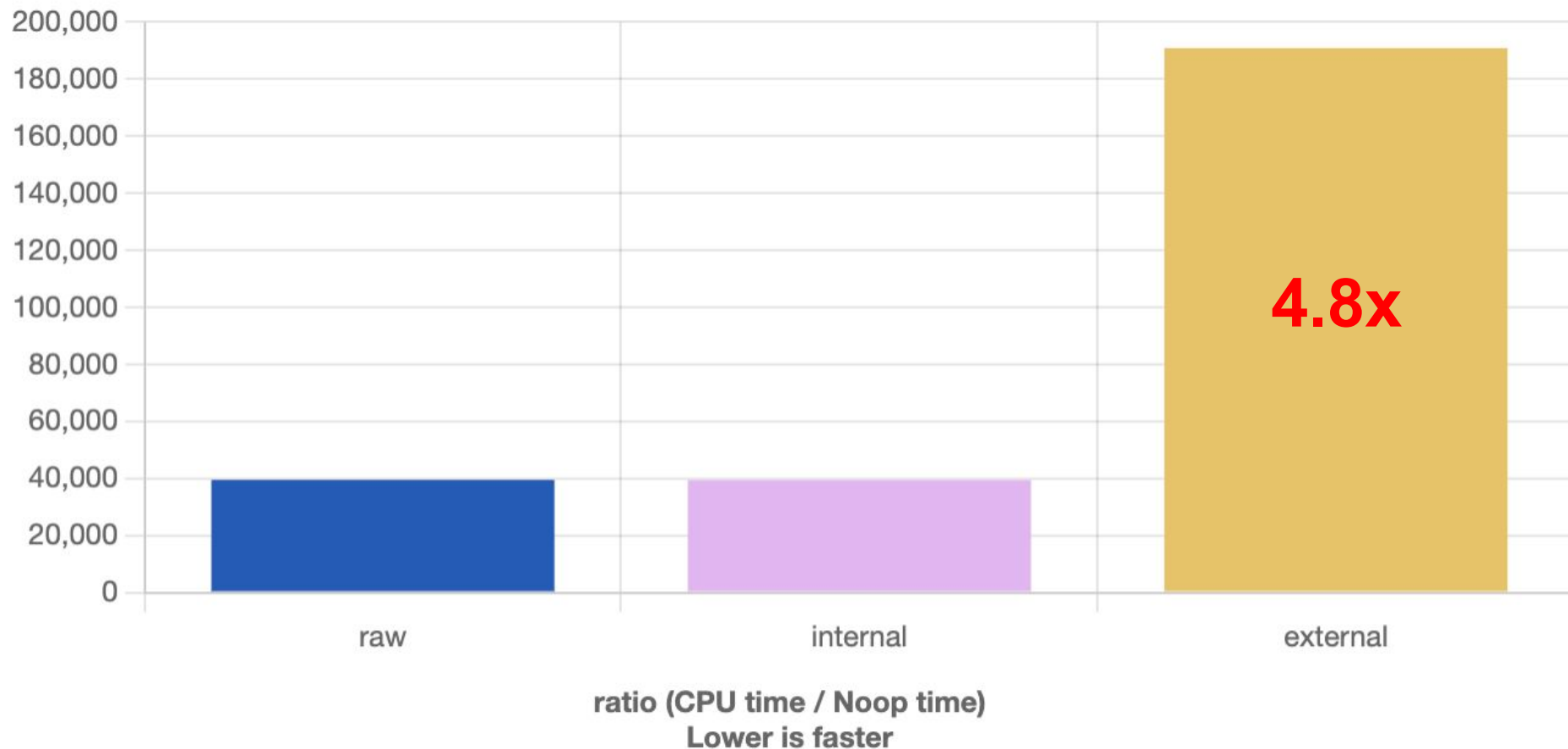
Raw loops	Internal iteration	External iteration
<pre> lea rsi, [rdi+40000] xor eax, eax .L3: mov edx, DWORD PTR [rdi] lea ecx, [rax+rdx] and edx, 1 cmovbe eax, ecx add rdi, 4 cmp rdi, rsi jne .L3 ret </pre>	<pre> lea rsi, [rdi+40000] xor eax, eax .L10: mov edx, DWORD PTR [rdi] lea ecx, [rax+rdx] and edx, 1 cmovbe eax, ecx add rdi, 4 cmp rdi, rsi jne .L10 ret </pre>	<pre> lea rdx, [rdi+40000] jmp .L16 .L27: add rdi, 4 cmp rdx, rdi je .L26 .L16: mov eax, DWORD PTR [rdi] test al, 1 jne .L27 xor ecx, ecx cmp rdx, rdi je .L14 .L20: add ecx, eax jmp .L25 .L28: mov eax, DWORD PTR [rdi] test al, 1 je .L18 .L25: add rdi, 4 cmp rdx, rdi jne .L28 .L14: mov eax, ecx ret .L18: cmp rdi, rdx jne .L20 jmp .L14 .L26: xor ecx, ecx mov eax, ecx ret </pre>



filter performance, consecutive inline: quick-bench.com

41





cartesian_product

```
using Array = int const (&)[100];
```

```
int raw(Array l, Array r) {  
    int sum = 0;  
    for (int x : l)  
        for (int y : r)  
            sum += x ^ y;  
    return sum;  
}
```

```
int internal(Array l, Array r) {  
    int sum = 0;  
    for_each_while(  
        cartesian_product(l, r),  
        [&](auto tuple) {  
            auto [x, y] = tuple;  
            sum += x ^ y;  
            return true;  
        })  
    );  
    return sum;  
}
```

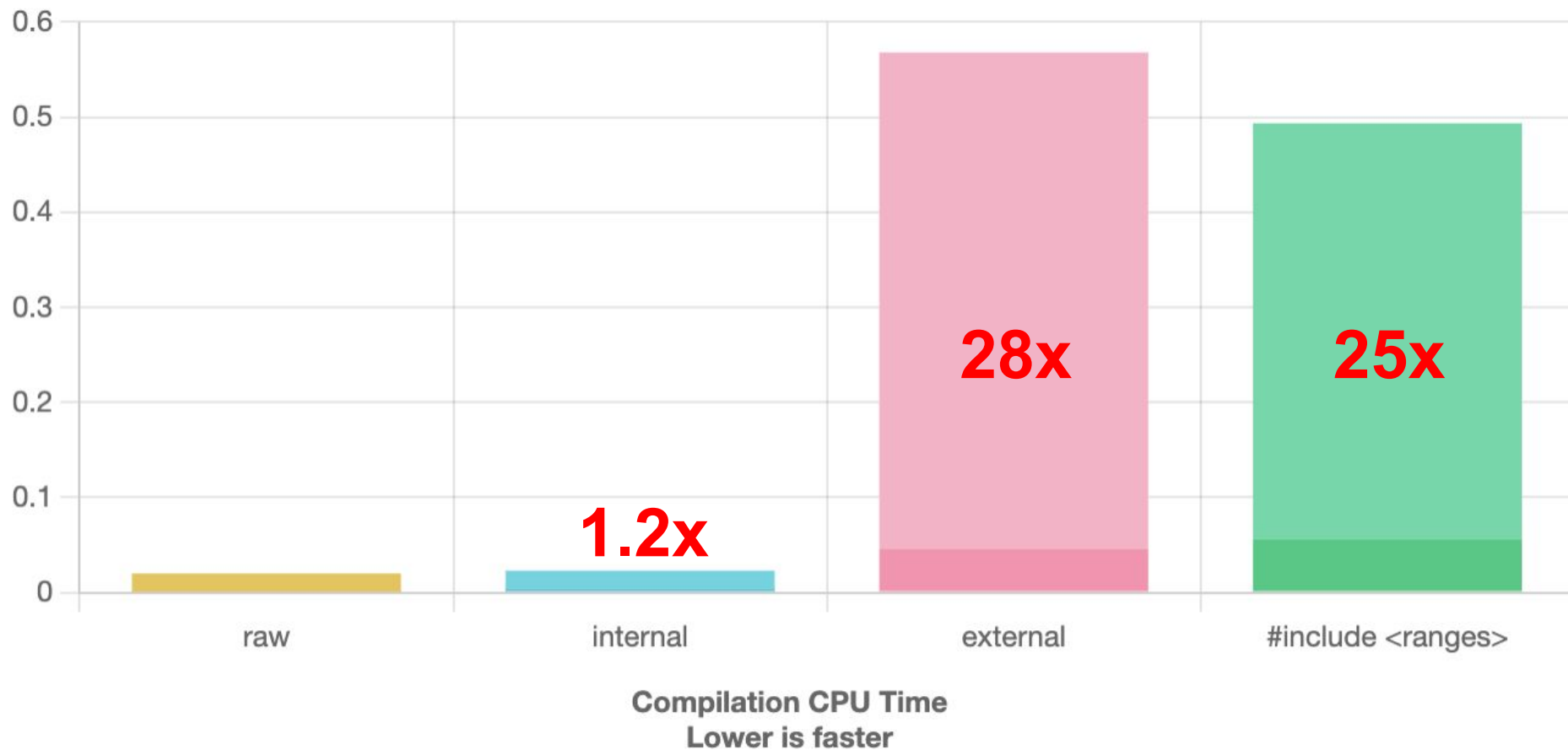
```
#include <ranges>  
  
int external(Array l, Array r) {  
    int sum = 0;  
    using std::views::  
        cartesian_product;  
    for (auto [x, y] :  
        cartesian_product(l, r)  
    )  
        sum += x ^ y;  
    return sum;  
}
```

cartesian_product implementation (limited)

```
1 template<typename First, typename Second>
2 struct cartesian_product {
3     First first;
4     Second second;
5
6     friend bool for_each_while(cartesian_product range, auto&& sink) {
7         return for_each_while(range.first,
8             [&](auto&& x) {
9                 return for_each_while(range.second, [&](auto&& y) {
10                     return sink(pair{x, y});
11                 });
12             }
13         );
14     }
15 };
```

vs 508 lines [range/v3/view/cartesian_product.hpp](#)

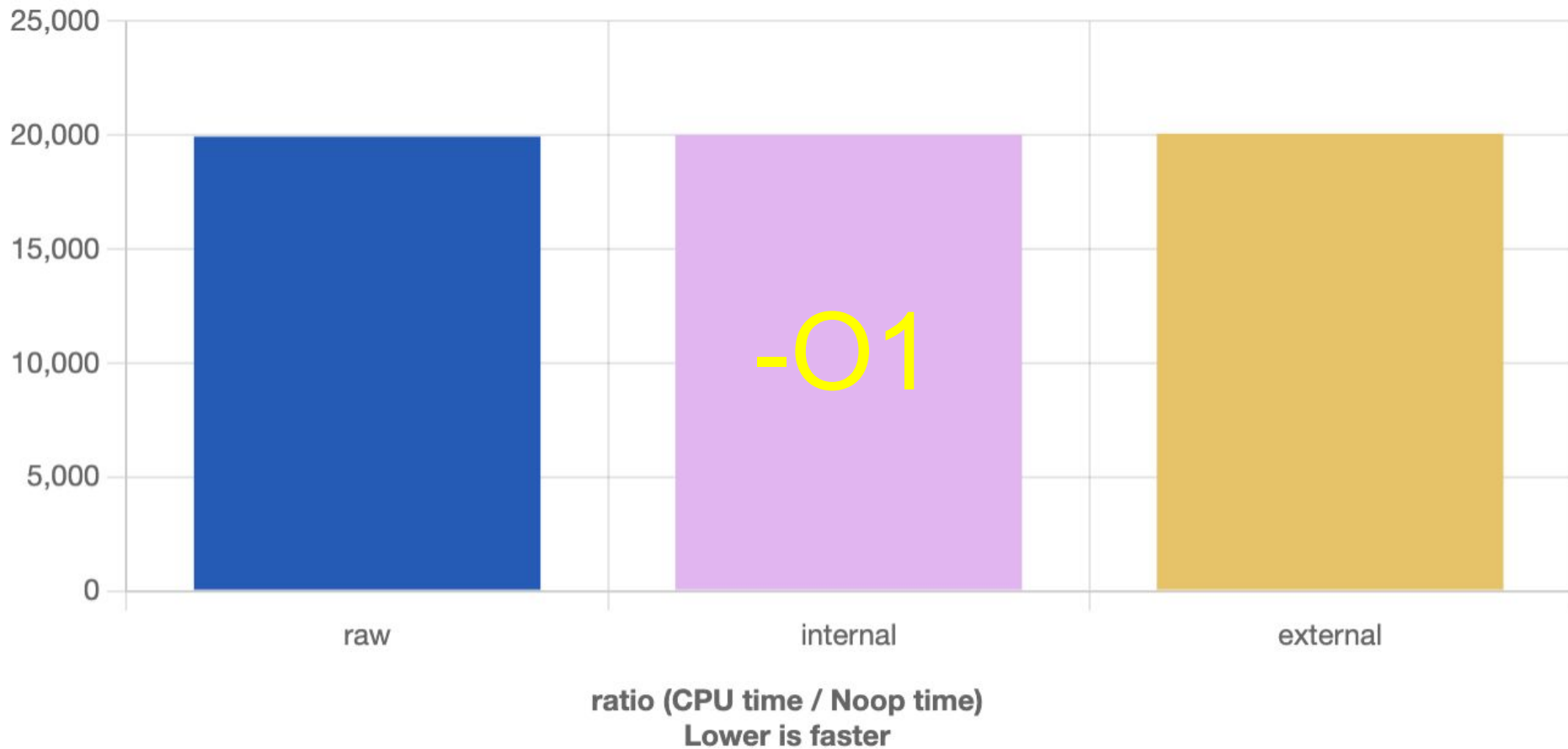
where iterator is forced to store `std::tuple<iterator_t<Inputs>...>`
and increment it as a big integer in mixed base



cartesian_product assembly: godbolt.org

46

Raw loops	Internal iteration	External iteration
<pre>mov r9, rdi mov r10, rsi mov r8, rdi add r9, 400 mov edx, 0 lea rdi, [rsi+400] .L3: mov esi, DWORD PTR [r8] mov rax, r10 .L2: mov ecx, esi xor ecx, DWORD PTR [rax] add edx, ecx add rax, 4 cmp rax, rdi jne .L2 add r8, 4 cmp r8, r9 jne .L3 mov eax, edx ret</pre>	<pre>mov r9, rdi mov r10, rsi mov r8, rdi add r9, 400 mov edx, 0 lea rdi, [rsi+400] .L8: mov esi, DWORD PTR [r8] mov rax, r10 .L7: mov ecx, esi xor ecx, DWORD PTR [rax] add edx, ecx add rax, 4 cmp rax, rdi jne .L7 add r8, 4 cmp r9, r8 jne .L8 mov eax, edx ret</pre>	<pre>lea r8, [rdi+400] mov rax, rsi mov ecx, 0 lea r9, [rsi+400] .L12: cmp r8, rdi je .L16 .L13: mov edx, DWORD PTR [rdi] xor edx, DWORD PTR [rax] add ecx, edx add rax, 4 cmp rax, r9 jne .L12 add rdi, 4 mov rax, rsi jmp .L12 .L16: cmp rsi, rax jne .L13 mov eax, ecx ret</pre>

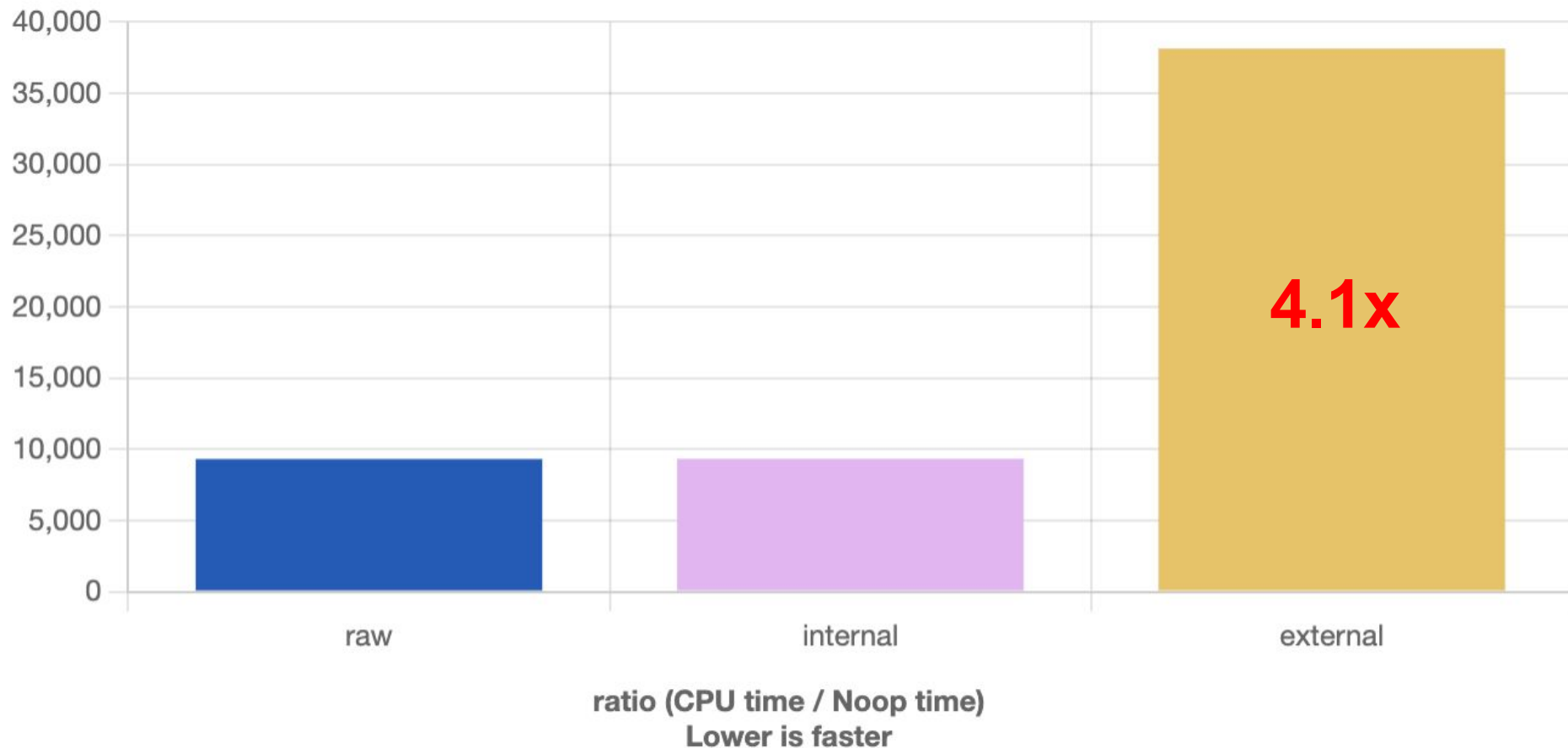


cartesian_product assembly, -O2: godbolt.org

48

Raw loops	Internal iteration	External iteration
<pre>mov rcx, rdi pxor xmm1, xmm1 lea rdi, [rdi+400] lea rdx, [rsi+400] .L3: movdqu xmm2, XMMWORD PTR [rcx] mov rax, rsi .L2: movd xmm3, DWORD PTR [rax] add rax, 4 pshufd xmm0, xmm3, 0 pxor xmm0, xmm2 paddb xmm1, xmm0 cmp rax, rdx jne .L2 add rcx, 16 cmp rcx, rdi jne .L3 movdqa xmm0, xmm1 psrldq xmm0, 8 paddb xmm1, xmm0 movdqa xmm0, xmm1 psrldq xmm0, 4 paddb xmm1, xmm0 movd eax, xmm1 ret</pre>	<pre>mov rcx, rdi pxor xmm1, xmm1 lea rdi, [rdi+400] lea rdx, [rsi+400] .L9: movdqu xmm2, XMMWORD PTR [rcx] mov rax, rsi .L8: movd xmm3, DWORD PTR [rax] add rax, 4 pshufd xmm0, xmm3, 0 pxor xmm0, xmm2 paddb xmm1, xmm0 cmp rax, rdx jne .L8 add rcx, 16 cmp rdi, rcx jne .L9 movdqa xmm0, xmm1 psrldq xmm0, 8 paddb xmm1, xmm0 movdqa xmm0, xmm1 psrldq xmm0, 4 paddb xmm1, xmm0 movd eax, xmm1 ret</pre>	<pre>lea r8, [rdi+400] mov rax, rsi xor ecx, ecx lea r9, [rsi+400] .L13: cmp rdi, r8 je .L17 .L14: mov edx, DWORD PTR [rdi] add rax, 4 xor edx, DWORD PTR [rax-4] add ecx, edx cmp rax, r9 jne .L13 add rdi, 4 mov rax, rsi cmp rdi, r8 jne .L14 .L17: cmp rsi, rax jne .L14 mov eax, ecx ret</pre>

cartesian_product performance, -O2: quick-bench.com⁴⁹



concat

50

```
using Array = int const (&)[10000];
```

```
int raw(Array l, Array r) {  
    int sum = 0;  
    for (int x : l)  
        sum += x;  
    for (int x : r)  
        sum += x;  
    return sum;  
}
```

```
int internal(Array l, Array r) {  
    int sum = 0;  
    for_each_while(  
        concat(l, r),  
        [&](int x) {  
            sum += x;  
            return true;  
        })  
    );  
    return sum;  
}
```

```
#include  
<range/v3/view/concat.hpp>  
  
int external(Array l, Array r) {  
    int sum = 0;  
    using ranges::view::concat;  
    for (int x : concat(l, r))  
        sum += x;  
    return sum;  
}
```

concat implementation

```

1  template<typename... Inputs>
2  struct concat {
3      std::tuple<Inputs...> inputs;
4
5      concat(Inputs... inputs) : inputs{std::move(inputs)...} {}
6
7      friend bool for_each_while(concat range, auto&& sink) {
8          return std::apply(
9              [&](Inputs&... input) {
10                 return (... and for_each_while(input, sink));
11             },
12             range.inputs
13         );
14     }
15 };

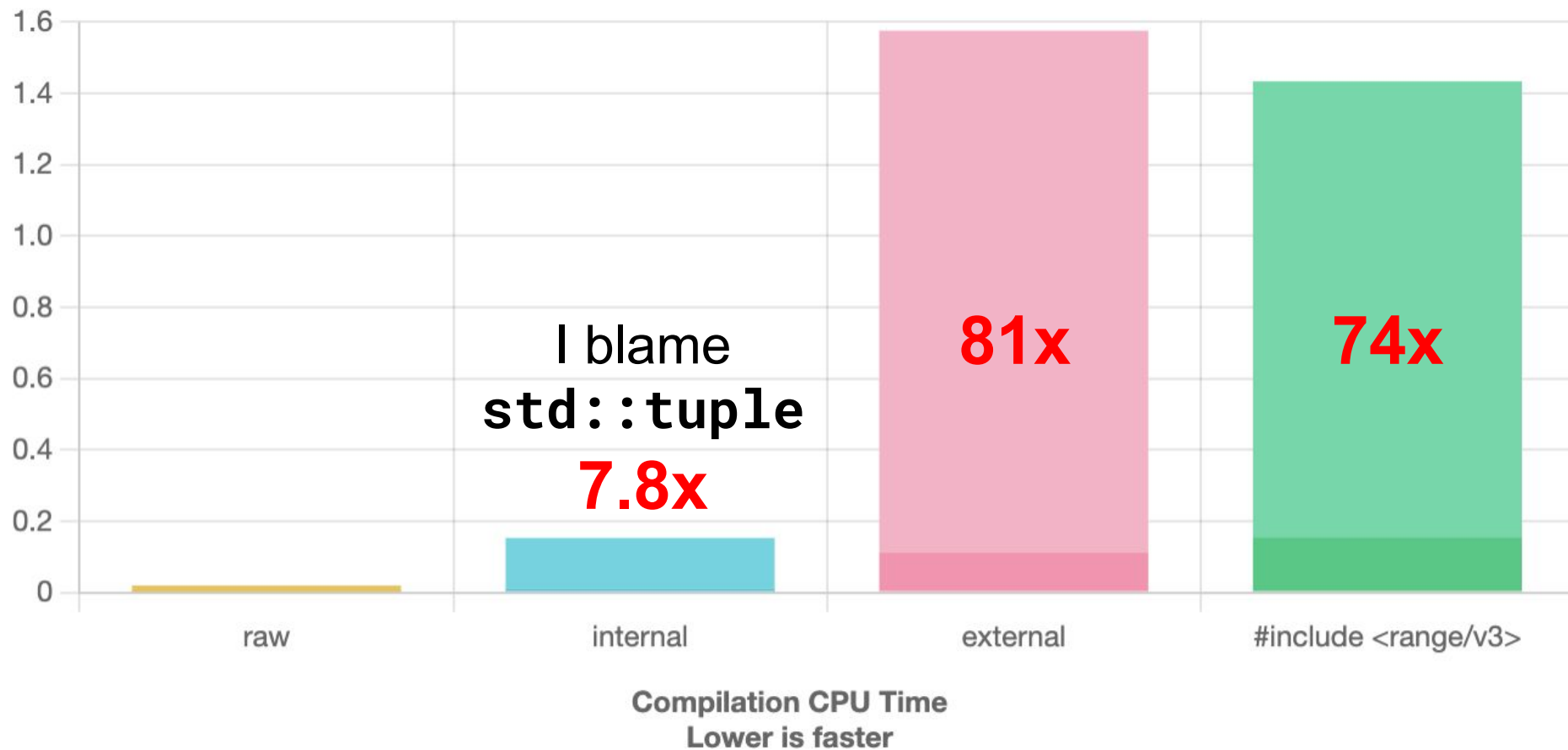
```

Fold expression defines
the control flow

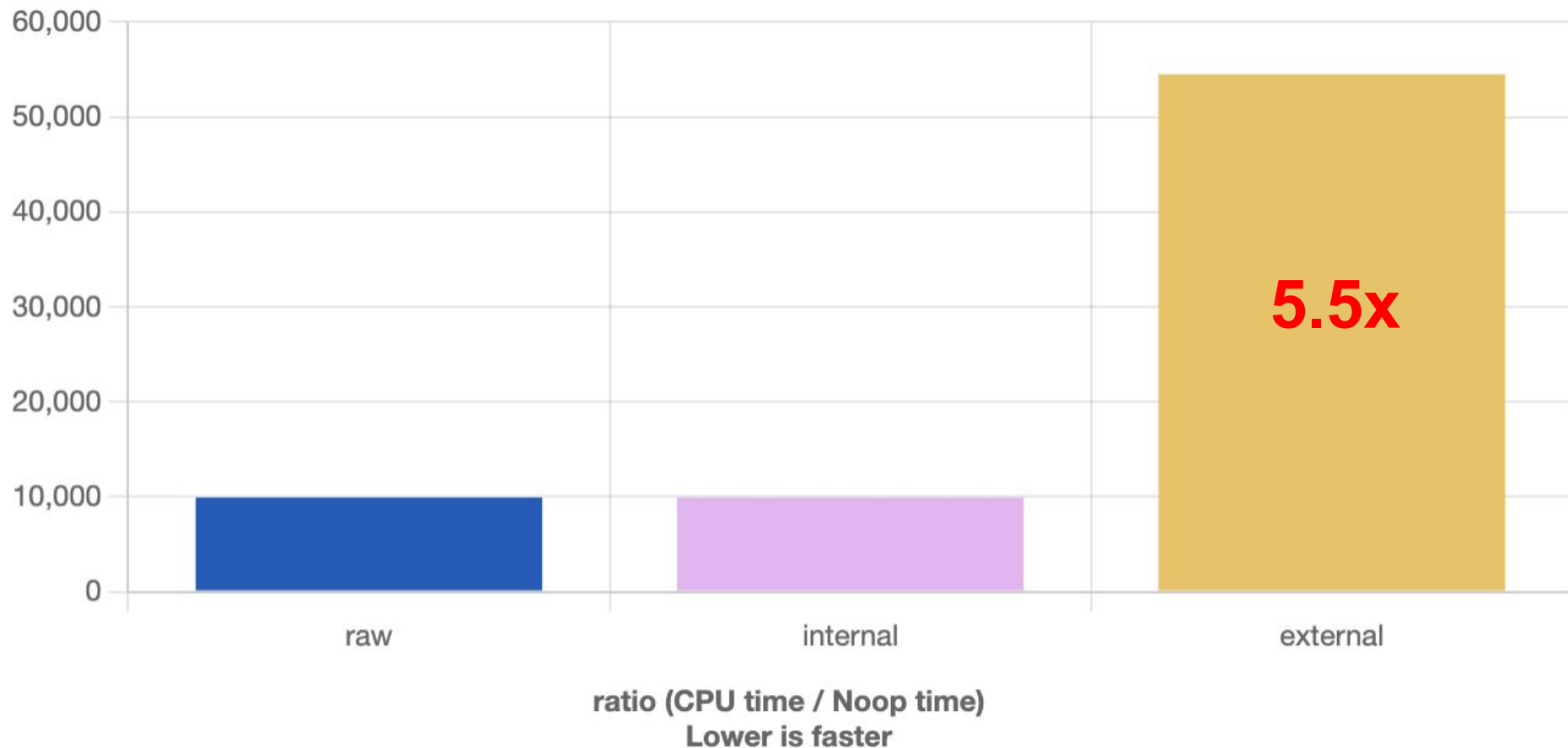
Parts of it are
delegated to inputs

vs 451 lines in [range/v3/view/concat.hpp](#)

where iterator is forced to store `std::variant<iterator_t<Inputs>...>`



Raw loops	Internal iteration	External iteration godbolt.org 53
<pre> lea rax, [rdi+40000] pxor xmm0, xmm0 .L14: movdqu xmm3, XMMWORD PTR [rdi] add rdi, 16 paddb xmm0, xmm3 cmp rdi, rax jne .L14 movdqa xmm1, xmm0 lea rax, [rsi+40000] psrldq xmm1, 8 paddb xmm0, xmm1 movdqa xmm1, xmm0 psrldq xmm1, 4 paddb xmm0, xmm1 movdqa xmm1, xmm0 pxor xmm0, xmm0 .L15: movdqu xmm4, XMMWORD PTR [rsi] add rsi, 16 paddb xmm0, xmm4 cmp rsi, rax jne .L15 movdqa xmm2, xmm0 psrldq xmm2, 8 paddb xmm0, xmm2 movdqa xmm2, xmm0 psrldq xmm2, 4 paddb xmm0, xmm2 paddb xmm0, xmm1 movd eax, xmm0 ret </pre>	<pre> lea rax, [rdi+40000] pxor xmm0, xmm0 .L19: movdqu xmm4, XMMWORD PTR [rdi] add rdi, 16 paddb xmm0, xmm4 cmp rax, rdi jne .L19 movdqa xmm1, xmm0 lea rax, [rsi+40000] psrldq xmm1, 8 paddb xmm0, xmm1 movdqa xmm1, xmm0 psrldq xmm1, 4 paddb xmm0, xmm1 movdqa xmm1, xmm0 pxor xmm0, xmm0 .L20: movdqu xmm3, XMMWORD PTR [rsi] add rsi, 16 paddb xmm0, xmm3 cmp rsi, rax jne .L20 movdqa xmm2, xmm0 psrldq xmm2, 8 paddb xmm0, xmm2 movdqa xmm2, xmm0 psrldq xmm2, 4 paddb xmm0, xmm2 paddb xmm0, xmm1 movd eax, xmm0 ret </pre>	<pre> push r13 lea r13, [rdi+40000] push r12 xor r12d, r12d push rbp mov rbp, rdi push rbx mov rbx, rsi sub rsp, 40 .L34: mov rsi, rsp lea rdi, [rsp+16] mov QWORD PTR [rsp], rbp mov QWORD PTR [rsp+8], 0 call ranges::variant_unique cmp QWORD PTR [rsp+24], 0 jne .L37 mov rax, QWORD PTR [rsp+16] add r12d, DWORD PTR [rax] cmp rbp, r13 je .L42 add rbp, 4 cmp r13, rbp jne .L34 lea rbp, [rbx+40000] jmp .L33 .L43: mov rsi, rsp lea rdi, [rsp+16] mov QWORD PTR [rsp], rbx mov QWORD PTR [rsp+8], 1 call ranges::variant_unique cmp QWORD PTR [rsp+24], 0 jne .L37 mov rax, QWORD PTR [rsp+16] add r12d, DWORD PTR [rax] cmp rbp, r13 je .L42 add rbp, 4 cmp r13, rbp jne .L34 lea rbp, [rbx+40000] jmp .L33 .L43: mov rsi, rsp lea rdi, [rsp+16] mov QWORD PTR [rsp], rbx mov QWORD PTR [rsp+8], 1 call ranges::variant_unique cmp QWORD PTR [rsp+24], 0 jne .L37 mov ecx, OFFSET FLAT:.LC7 mov edx, 342 mov esi, OFFSET FLAT:.LC1 mov edi, OFFSET FLAT:.LC2 call __assert_fail .L42: mov ecx, OFFSET FLAT:.LC8 mov edx, 137 mov esi, OFFSET FLAT:.LC9 mov edi, OFFSET FLAT:.LC10 call __assert_fail </pre>



1. Iterators vs indices
2. External vs internal iteration
 - a. Simplicity and performance
 - b. Composability
3. Internal iteration design space

RANGES

transform-filter

```
auto const inputs = std::vector<int>{1, 2, 3, 4, 5, 6};

std::vector<int> const results = inputs
| ranges::view::transform([](int i){ cout << i << '\n'; return i * 2; })
| ranges::view::filter([](int i){ return i % 3 == 0; });
```

```
1
2
3
4
5
6
```

7

ansatz

Jonathan Boccard



Pipes:
How Plumbing
Can Make Your
C++ Code More
Expressive

Cppcon.org

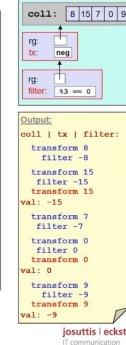


Pipelines: Price of Filter Views

```
std::vector<int> coll{ 8, 15, 7, 0, 9 };

// define a view on coll:
auto vColl = coll
| std::views::transform([](int i) {
    std::cout << " transform "
    << i << '\n';
    return -i;
})
| std::views::filter([](int i) {
    std::cout << " filter "
    << i << '\n';
    return i % 3 == 0;
});

// and use it:
std::cout << "coll | tx | filter:\n";
for (int val : vColl) {
    std::cout << "val: " << val << '\n';
}
```



56

Taming the C++ Filter View

Nicolai Josuttis

+24

C++ now 2024 Aspen, Colorado

CppNow.org

Video Sponsorship Provided By

millennium
think-cell



Rappel: Compose Algorithms, Not Iterators
Google's Alternative to Ranges

John Bandela, Chris Philip

Cppcon.org

Video Sponsorship Provided By

think-cell



transform | filter

```
1 println("t"),
2 vector{1, 2, 3, 4, 5}
3 | transform([](int n) { return n * 2; })
4 | filter([](int n) { return n % 4 == 0; })
5 |

1 auto t = {}(int n) {
2   println("transform({})", n);
3   return n * 2;
4 };
5 auto f = {}(int n) {
6   println("filter({})", n);
7   return n % 4 == 0;
8 };
9 println("t"),
10 vector{1, 2, 3, 4, 5}
11 | transform(t) | filter(f);

transform(1)
filter(1)
transform(2)
filter(2)
transform(3)
filter(3)
transform(4)
filter(4)
transform(5)
filter(5)
transform(6)
filter(6)
transform(7)
filter(7)
transform(8)
filter(8)
transform(9)
filter(9)
transform(10)
filter(10)
transform(11)
filter(11)
```

t(2) and t(4) are called twice!

Ranges++:
Are Output Range Adaptors
the Next Iteration of C++ Ranges?
Daisy Hollman

+24

The Terrible Problem Of Incrementing A Smart Iterator

```
1 #include <iostream>
2 #include <ranges>
3 #include <vector>
4
5 int times2(int n) {
6     std::cout << "transform " << n << '\n';
7     return n * 2;
8 }
9
10 bool isMultipleOf4(int n) {
11     return n % 4 == 0;
12 }
13
14 int main() {
15     std::vector<int> numbers = {1, 2, 3, 4, 5};
16     auto results = numbers | std::views::transform(times2)
17                             | std::views::filter(isMultipleOf4);
18     for (auto result : results) {
19         std::cout << result << '\n';
20     }
21     return 0;
22 }
```

fluentcpp.com
godbolt.org

```
transform 1
transform 2
transform 2
4
transform 3
transform 4
transform 4
8
transform 5
```

The Terrible Problem Of Incrementing A Smart Iterator

58

```
auto last = end(range);  
auto iter = begin(range);  
while (iter != last) {  
    process(*iter);  
    ++iter;  
}
```

```
auto last = end(numbers);  
auto iter = begin(numbers);  
while (iter != last) {  
    if (isMultipleOf4(times2(*iter)))  
        break;  
    ++iter;  
}  
  
while (iter != last) {  
    std::cout << times2(*iter) << '\n';  
    ++iter;  
    while (iter != last) {  
        if (isMultipleOf4(times2(*iter)))  
            break;  
        ++iter;  
    }  
}
```


Not a problem for internal iteration

```
1 template<typename Input, typename Predicate>
2 struct filter_view {
3     Input input;
4     Predicate filter;
5
6     friend bool for_each_while(filter_view view, auto&& sink) {
7         return for_each_while(view.input, [&](auto&& x) {
8             if (view.filter(x))
9                 return sink(x);
10            else
11                return true; // continue
12        });
13    }
14 };
```

Value passed here is already transformed, and it's alive for the whole duration of a function call

It can be reused without recomputing

Moreover, we're free to create other temporaries needed by the loop

reverse has a dual terrible problem

Example by Nicolai Josuttis from [P3725](#):

```
1 std::vector<std::string> coll1{"Amsterdam", "Berlin", "Cologne", "LA"};
2
3 // move long strings in reverse order to another container:
4 auto large = [](auto const& s) { return s.size() > 5; };
5 auto sub = coll1 | std::views::filter(large)
6                  | std::views::reverse
7                  | std::views::as_rvalue
8                  | std::ranges::to<std::vector>();
```

reverse has a dual terrible problem

```
auto iter = begin(range);  
auto last = end(range);  
while (iter != last) {  
    process(*iter);  
    ++iter;  
}
```

reverse: duplicates decrement
filter: -- has no bound check

```
std::vector<std::string> result;  
auto iter = end(coll1);  
auto last = begin(coll1);  
while (last != iter  
        && !large(*last))  
    ++last;  
while (iter != last) {  
    auto iter_tmp = iter - 1;  
    while (!large(*iter_tmp))  
        --iter_tmp;  
    result.push_back(  
        std::move(*iter_tmp));  
    --iter;  
    while (!large(*iter))  
        --iter;  
}
```

Nicolai Josuttis on constness, ownership, caching

63



Cppcon.org



C++20: Views

C++20

```
void print(const std::ranges::input_range auto& coll)
{
    for (const auto& elem : coll) {
        std::cout << elem << ' ';
    }
    std::cout << '\n';
}

std::vector<int> coll1{0, 8, 15, 47, 11, 42, 1};
std::set<int> coll2{0, 8, 15, 47, 11, 42, 1};

print(coll1);
print(coll2);

print(std::views::take(coll1, 3)); // print first three elements
print(std::views::take(coll2, 3)); // print first three elements
print(coll1 | std::views::take(3)); // print first three elements
print(coll2 | std::views::take(3)); // print first three elements

print(coll2 | std::views::take(3)
  | std::views::transform([](auto v) {
    return std::to_string(v) + 's';
}));
```

Output:

```
0 8 15 47 11 42 1
0 1 8 11 15 42 47
0 8 15
0 1 8
0 8 15
0 1 8
0s 1s 8s
```

C++

©2024 by josuttis.com

6

josuttis | eckstein
IT communication

Taming the C++ Filter View

Nicolai Josuttis



Printing a range with a delimiter

```
1 void print(const std::ranges::input_range auto& coll) {  
2     for (const auto& elem : coll) {  
3         std::cout << elem << ' '; extra trailing space  
4     }  
5     std::cout << '\n';  
6 }
```

Not composable!

Solution using “raw loops”

```
1  void print(std::ranges::input_range auto const& coll) {
2      bool is_first = true;
3      for (auto const& elem : coll) {
4          if (is_first)
5              is_first = false;
6          else
7              std::cout << ' ';
8          std::cout << elem;
9      }
10 }
```

Solution using internal iteration

```
1 template<typename Input, typename Delimiter>
2 struct intersperse {
3     Input input;
4     Delimiter delimiter;
5
6     friend void for_each_while(intersperse view, auto&& sink) {
7         bool is_first = true;
8         return for_each_while(view.input, [&](auto const& x) {
9             if (is_first)
10                 is_first = false;
11             else
12                 sink(delimiter);
13             sink(x);
14         });
15     }
16 };
```

Making it composable: fluentcpp.com

```
1 // inputs:    [x1, x2, ... xn]
2 // transform: [[x1, d], [x2, d], ... [xn, d]]
3 // flatten:   [ x1, d, x2, d, ... xn, d ]
4 // drop last: [ x1, d, x2, d, ... xn ]
5 auto intersperse_view =
6 view::transform([delim](auto inp)
7 {
8     return std::array<decltype(inp), 2>{{ std::move(inp), delim }};
9 })
10 | view::join
11 | view::drop_last(1);
```

view::join doesn't accept rvalue containers.

But it does accept rvalue views!

view::drop_last requires SizedRange or ForwardRange input

view::join returns an InputRange (even if its input is a ForwardRange)

Solution by Tristan Brindle: [reddit.com](https://www.reddit.com)

```
1 namespace views = ranges::views;  
2  
3 auto intersperse = [](auto delimiter) {  
4     auto make_pair = [=](auto value) {  
5         return views::concat(views::single(delimiter), views::single(value)); };  
6     return views::transform(make_pair) | views::join | views::drop(1);  
7 };
```

1. Iterators vs indices
2. External vs internal iteration
 - a. Simplicity and performance
 - b. Composability
3. Internal iteration design space

External iteration design space: [youtube.com](https://www.youtube.com/watch?v=8m33333333)

Iterators and Ranges: Comparing C++ to D, Rust, and Others



The Rust/Python/Java/... Iterator Model

	read	advance	done?
	<code>*it</code>	<code>++it</code>	<code>it == last</code>
	<code>r.front()</code>	<code>r.popFront()</code>	<code>r.empty()</code>
	<code>e.Current()</code>	<code>e.MoveNext()</code>	
		<code>it.next()</code>	
		<code>it.__next__()</code>	
	<code>it.next()</code>		<code>it.hasNext()</code>



Barry Revzin



CODE RECKONS

Science to the CORE

Internal iteration core APIs

for_each_while(range, sink)

sink input	sink output	for_each_while output
1. Element reference 2. Element iterator: for niche use cases transrangers: github.com	1. bool 2. void: no short circuiting 3. More than 2 states: lexicographical_compare 4. continue_ / break_: intent is more explicit think-cell-library: github.com daisychains: github.com	1. Output of last sink call 2. Index: internal iteration becomes find_if Flux: github.com Cons: index implementation complexity is back

Iteration context with re-entry (pull API): youtube.com



Faster, Safer, Better Ranges

think-cell 



Tristan Brindle

Iteration context example 2

```
template <std::ranges::forward_range R>
struct range_iteration_context : immovable {
    std::ranges::iterator_t<R> iter;
    std::ranges::sentinel_t<R> const sent;

    range_iteration_context(R& rng)
        : iter(std::ranges::begin(rng)),
          sent(std::ranges::end(rng))
    {}

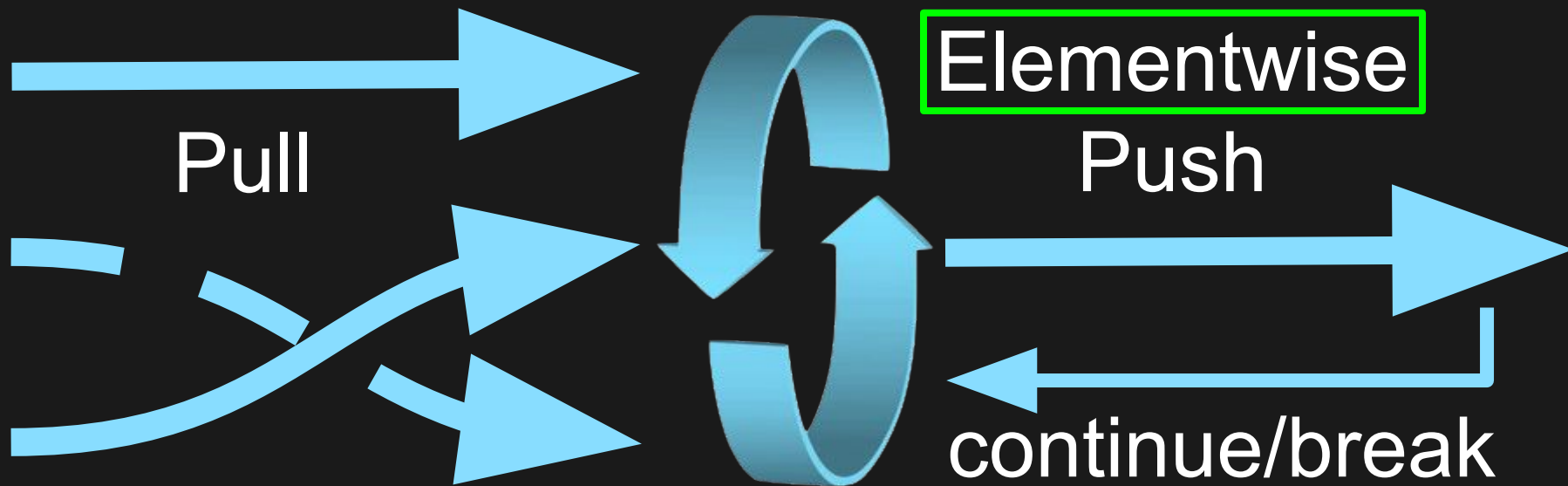
    using element_type = std::ranges::range_reference_t<R>;

    iteration_result run_while(auto&& pred)
    {
        while (iter != sent) {
            if (!pred(*iter++)) {
                return iteration_result::incomplete;
            }
        }
        return iteration_result::complete;
    }
};
```

Also found in:

- rivers: github.com
- transrangers: github.com

Control flow (loop)



Some views work elementwise: **transform**, **filter**

```
1 template<typename Predicate>
2 struct filter_pipe : pipe_base {
3     template<typename Value, typename TailPipeline>
4     void onReceive(Value&& value, TailPipeline&& tailPipeline) {
5         if (predicate_(value))
6             send(FWD(value), tailPipeline);
7     }
8
9     Predicate predicate_;
10 };
```

Takes a single element, no loop needed

Stores only the predicate, no input range or sink

Similar concept is found as:

- “pipe” in pipes: github.com
- “sink” in think-cell-library: github.com
- “link” in daisychains: github.com
- “incremental processing style” in SPL: github.com

Limitations of internal iteration

1. Cannot be used in a range-based for loop

[P2881](#) by Jonathan and Barry tried to address this, but got rejected

2. **for_each_while** cannot be called on 2+ ranges at the same time:

zip, equal, lexicographical_compare

Iteration context seems to solve this

3. **reverse** needs to reorder elements
4. Indices are sometimes necessary:
 - a. Finding a position where to insert or erase elements
 - b. Referencing sub-ranges: **split, chunk, slide**

LIMITATIONS

Pipes don't know what's coming next. **How to implement reverse?** drop_last?
 Pipes don't cache input data. How to implement split?
 Can't send a range to an individual pipe: inputs >>= transform(f);
 Have to create the results container on a separate statement

STRENGTHS

transform called only once per element
 rvalues accepted as inputs
 Branching out: tee, demux, unzip
 Pick up results from STL algorithms
 Multiple inputs without tuples
 Expressive insertion in containers: map_aggregator, etc.
 Easy to implement

59

ansatz



Jonathan Boccard

Pipes:
 How Plumbing
 Can Make Your
 C++ Code More
 Expressive

Cppcon.org



Design Alternatives for Filter Views

- begin () is cached**
 - Severe compile-time errors: unresolvable with const
 - Runtime errors: reading is trivial mod
- begin () is initialized du**
 - Performance issue: Initiali
- begin () is thread safe**
 - Performance issue: Makes
- No caching at all**
 - Performance issue: Some
 - Reverse view should cache ins
 - Programmers can and have to use
- No caching and filter iterators become input iterators**
 - Disables algorithms with multiple or reverse iterations
 - Some non-trivial use cases like with reverse no longer compile

Possible path to go ?

- All described issues are gone
 - Can read iterate
 - Read is thread safe
 - empty () is stateless
 - Can write via iteration
 - Can write between iterations
- What compiles is backward compatible
 - Bad performance does not compile

C++

©2024 by josuttis.com

84

josuttis | eckstein
 IT communication

Taming the C++ Filter View

Nicolai Josuttis

24

C++ now 2025
 Aspen, Colorado

CppNow.org

Video Sponsorship Provided By

think-cell

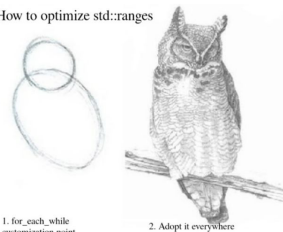
Bloomberg

Engineering



The rest of the owl

How to optimize std::ranges



1. for_each_while customization point
 https://ericniebler.com/

2. Adopt it everywhere

- stdv::reverse:**
 std2r::for_each_while_reversed?
- stdv::zip:** Can only use
 std2r::for_each_while once, all
 other ranges use iterators, so which one
 should use it?
- stdr::find and variants:**
 std2r::for_each_iterator_while?
- Generators:** Ranges that only support
 std2r::for_each_while but don't
 have iterators?
- Optimizations ignore range-based for
 loop**

think-cell

Jonathan Müller — @fsonathan

Missing (and Future?) C++ Range Concepts

C++ Now 2025-05-01

33

Missing (and Future?) C++ Range Concepts

Jonathan Müller

Highlighted
 by
 think-cell

Cppcon.org

Video Sponsorship Provided By

think-cell



What have we learned about output iterator composition?

- Output iterator composition is an *"inside-out" version* of the views in std::ranges
- Both push and pull models *can be lazy*
- Push models *work around TPOIASI* by merging the output iterator analogs of the increment and dereference operations into *one customization point*
 - But this simply doesn't work for some use cases. For instance:
 - reverse
 - sort (or anything else that's multipass)
 - slide (can be done, but it's awkward)
 - There are also some use cases that simply don't work with a pull model
 - Flattened one-to-many transforms (e.g., flat_map; can be done, but it's a lot more awkward)
- Push-style composition *can exist alongside* pull-style composition
 - This is a good thing! It means we *could have both* in the standard library one day

Ranges++:
 Are Output Range Adaptors
 the Next Iteration of C++ Ranges?

Daisy Hollman

24

Fall back to indices or iterators

C++ now

2023
MAY 8-12
Aspen, Colorado, USA



Jonathan Müller

The New Library on the Block
*A Strong Library Foundation
for Your next Project*

tc::for_each

```
tc::for_each(rng, sink)
```

Tries in order:

- 1 sink.chunk(rng) for appender
- 2 ADL-based customization point for_each_impl(rng, sink)
- 3 rng(sink) for generator ranges
- 4 Index based iteration.
- 5 Iterator based iteration.

think-cell

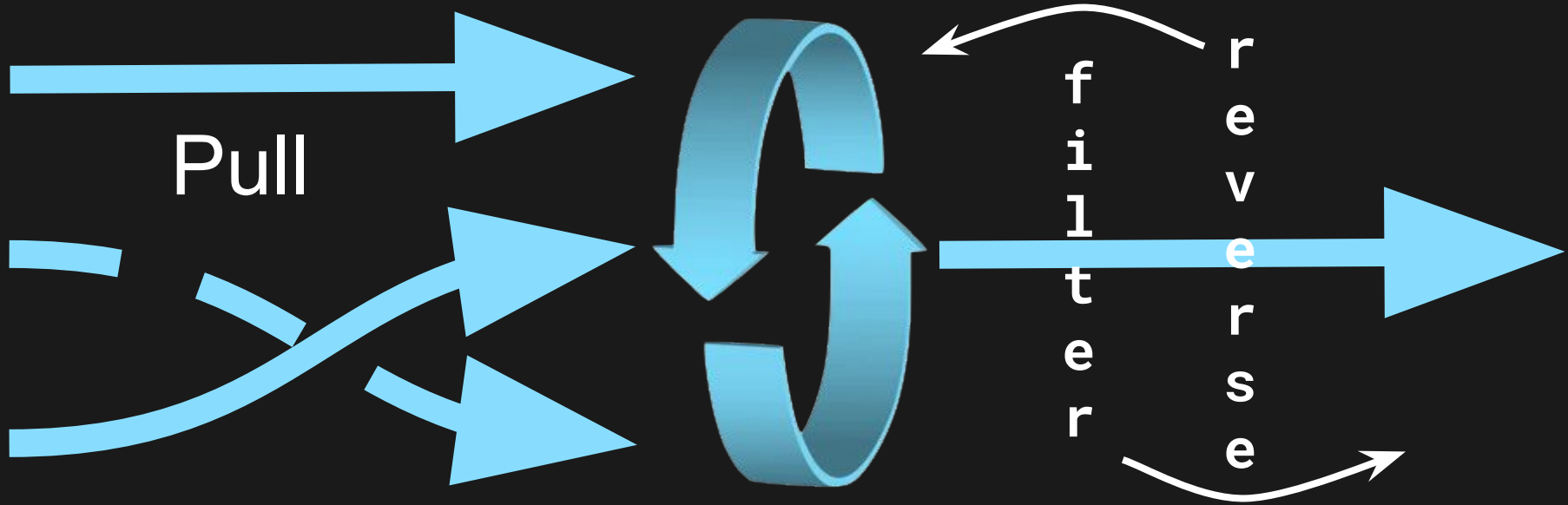
Stop being lazy!

Stop being lazy!

```
1 struct reverse
2 {
3     template<typename Gen>
4     auto operator()(seq<Gen> r) const -> std::vector<typename seq<Gen>::value_type>
5     {
6         auto vec = to_vector{}(std::move(r));
7         std::reverse(vec.begin(), vec.end());
8         return vec;
9     }
10    ...
11 };
```

- rangeless does it: [github.com](https://github.com/rangeless)
- SPL does it for `sort` etc: [github.com](https://github.com/rangeless)
- Linear algebra libraries typically evaluate parameters for matrix multiplication

Control flow (loop)



reverse problem

```
auto view = input  
    | filter(is_even)  
    | reverse  
    | ...
```

reverse problem solved

```
auto view = input
```

```
| reverse
```

```
| filter(is_even)
```

```
| ...
```

Simplify, revisited (and)

```
1: template <matcher L, matcher R>
2: constexpr auto simplify(and_t<L, R> const &m) {
3:     auto l = simplify(m.lhs);
4:     auto r = simplify(m.rhs);
5:
6:     if constexpr (implies(l, r)) {
7:         return l;
8:     } else if constexpr (implies(r, l)) {
9:         return r;
10:    } else if constexpr (implies(l, negate(r)) or implies(r, negate(l))) {
11:        return never;
12:    } else {
13:        return and_t{l, r};
14:    }
15: }
```

This is what it looks like now.

77 / 98

Message Handling with Boolean Algebra

Ben Deane

Equivalent transformations of nested views

```
1 template<typename Predicate, typename Input>
2 auto simplify(reverse_view<filter_view<Predicate, Input>> view) {
3     return std::move(view.input.input)
4         | reverse
5         | filter(view.input.predicate);
6 }
```

Similarly, any order changing operation (like permutation) can be swapped with any elementwise (like transform)

Other useful transformations

- `transform | drop` $\rightarrow$ `drop | transform`
- `transform | take` $\rightarrow$ `take | transform`
- `transform | stride` $\rightarrow$ `stride | transform`
- `transform(f1) | transform(f2)` $\rightarrow$ `transform(f2(f1))`
- `filter(p1) | filter(p2)` $\rightarrow$ `filter(p1 and p2)`
- `reverse | reverse` $\rightarrow$ `identity`

Drawbacks

- Uncommon technique not available in STL, so it requires teaching
- Simplified type is different from the original, which might be confusing
- For N view types, there are N^2 combination pairs to handle
- Generic concept-based simplification rules might overlap
- Not all combinations can be simplified
- Doesn't work with non-movable types

Expressive power to complexity ratio

1. Containers with indices
2. Internal iteration
3. Views with indices for simple cases like
transform, take, drop
4. Equivalent transformations of nested views

Call to action

1. When designing abstractions – check how well they model the problem
2. Don't use `std::ranges::views`

Range algorithms are fine, just not composable

3. If you're looking for alternatives:
 - a. Flux for production.
 - b. rivers for experimentation.
4. Allocate resources to standardize internal iteration instead of adding more views
5. Discuss a path to switch standard containers and algorithms from iterators to indices (random-access containers already have them)
6. Discuss designs to define code transformation rules in code

Thank you for attention!

Indices for node-based containers

- Pointer is effectively an address, which is an index into the main memory
- Main memory is a singleton, and pointer dereference $*p$ relies on that
- Node-based containers of size $> 2^{32}$ are very rare in practice
- We could associate a smaller memory region with a container
- Then define index as an address into this memory region, which can have smaller size than the pointer
- Read operation works by combining index with the memory region for the container